

Partial Differential Equation Agents (PDE-Agents): A Large Language Model-Orchestrated Multi-Agent Framework for Automated Finite Element Simulations with Knowledge Graph-Augmented Reasoning

Anonymous^a

^a[Removed for review],

Abstract

We present *PDE-Agents*, a multi-agent ecosystem that automates the full lifecycle of partial differential equation (PDE) / finite element method (FEM) simulations through natural-language interaction. The system combines three specialist large language model (LLM) agents—Simulation, Analytics, and Database—orchestrated via a LangGraph supervisor graph, with a locally-deployed open-source LLM stack (Qwen3-Coder-Next, Llama 4 Scout) running on dual NVIDIA RTX PRO 6000 Blackwell graphics processing units (GPUs) with approximately 196 gigabytes of combined video memory. The architecture is model-agnostic; we report cross-model validation across two generations of open-source LLMs. A central novelty is the integration of a *graph-based retrieval-augmented generation* (GraphRAG) knowledge base (Neo4j + 768-dimensional vector embeddings) that enriches agents with curated material properties, known failure patterns, and prior run lineage. We report seven empirical contributions: (i) a formal verification and validation (V&V) study confirming second-order spatial convergence ($\mathcal{O}(h^2)$) for two non-trivial benchmark cases and algebraic exactness for the linear case on

the heat equation solver; (ii) a three-way ablation study across 50 benchmark tasks with a frozen knowledge graph (KG), comparing KG On, KG Off, and KG Smart, showing that KG Smart achieves 100% success *and* the highest output quality (physics score 0.933 vs. 0.853 for KG Off, material property fidelity (MPF) 0.926 vs. 0.796), and that our *KG Smart* integration, combining warm-start injection with lazy conditional retrieval, matches KG Off reliability while maximising output fidelity; (iii) a novel-material experiment using three fictional materials whose properties exist only in the KG, where KG Smart achieves near-perfect $\text{MPF} = 1.00$ versus 0.34 for the KG-free baseline; (iv) a failure analysis tracing KG On’s 3 systematic failures to budget exhaustion and timeout, establishing warm-start injection as the dominant factor in KG Smart’s reliability advantage; (v) an adaptive KG decision framework (Algorithm 1) that selects the optimal retrieval mode per task; (vi) production-scale agent quality metrics from 1,369 real simulation runs showing 97.8% overall success and an 85.4% first-try rate; and (vii) a controlled 100-task KG growth experiment showing that accumulated run history yields a difficulty-dependent quality gain, with hard-task MPF improving by 8.8% between passes while easy and novel tasks remain at ceiling. All code, models, and evaluation artifacts are released openly. Taken together, these results point to *integration pattern*, rather than knowledge content, as what decides whether GraphRAG augmentation helps or hinders an LLM agent, and they yield concrete design principles for autonomous FEM simulation assistants.

Keywords: large language model agents, finite element method, knowledge graph, graph-based retrieval-augmented generation, multi-agent systems, scientific computing automation

1. Introduction

Finite element method simulations are central to engineering analysis across structural mechanics, heat transfer, fluid dynamics, and electromagnetics. Yet the gap between a domain expert’s intent—“what if I use aluminium instead of steel?”—and a running, verified simulation involves a sequence of error-prone manual steps: geometry creation, boundary condition specification, solver parameter selection, result interpretation, and iterative debugging.

The recent emergence of *reasoning-capable* LLMs [1, 2, 3] has opened the possibility of agents that can autonomously traverse this workflow. Prior work on “AI for scientific computing” has largely focused on surrogate modelling [4, 5, 6], operator learning [7], or dataset-specific fine-tuning [8]. Much less attention has been paid to *procedural automation*—using LLMs to *set up and manage* solvers rather than to replace them.

Meanwhile, the *retrieval-augmented generation* (RAG) paradigm [9] has demonstrated that grounding LLM outputs with authoritative external knowledge dramatically reduces hallucination. The graph-structured variant, GraphRAG [10], further enables multi-hop reasoning over relational knowledge—particularly relevant in engineering contexts where material properties, mesh requirements, and solver stability rules form a rich, interconnected knowledge base.

In this paper we make the following contributions:

1. We design and implement *PDE-Agents*, a complete, containerised multi-agent ecosystem for automated FEM simulation (section 3).
2. We describe a novel *GraphRAG integration pattern* that encodes material properties, known failure modes, and prior run lineage as a Neo4j

- 26 knowledge graph with Hierarchical Navigable Small World (HNSW)
27 vector-indexed embeddings [11] (section 4).
- 28 3. We conduct a rigorous V&V study that confirms $\mathcal{O}(h^2)$ spatial conver-
29 gence for two non-trivial benchmark cases (transient Fourier and steady
30 Poisson) and algebraic exactness for the linear profile case, validating
31 the numerical kernel against analytical solutions (section 5).
- 32 4. We run a three-way ablation study (KG On, KG Off, KG Smart) over
33 50 tasks with a frozen knowledge graph. KG-enabled modes produce
34 higher-quality simulation outputs (physics score, material property
35 fidelity) than the KG-free baseline, and *KG Smart* reaches 100% success
36 and the highest output quality (section 6).
- 37 5. We introduce a *novel-material experiment* using three fictional materials
38 absent from any LLM training corpus. KG Smart attains $\approx 2.9\times$ the
39 material property fidelity of the KG-free baseline, which fabricates
40 physically incorrect properties (section 6.4).
- 41 6. We trace KG On’s 3 systematic failures to budget exhaustion and
42 timeout, revealing that warm-start injection, not lazy retrieval, is the
43 dominant factor in KG Smart’s reliability advantage (section 6.5).
- 44 7. We report production-scale agent quality metrics derived from 1,369 real
45 simulation runs, together with a controlled 100-task KG growth experi-
46 ment in which the quality gain proves difficulty-dependent: hard-task
47 MPF improves by 8.8% as the KG accumulates run history (section 7).

48 2. Related Work

49 **LLM agents for scientific computing:** ChemCrow [12] demonstrated
50 that equipping an LLM with chemistry tools (RDKit, reaction planners)
51 enables autonomous laboratory-scale reasoning. SciAgent [13] and similar
52 systems extend this to broader scientific question answering. Our work
53 targets *numerical simulation* rather than knowledge retrieval, requiring tool-
54 call pipelines that produce and verify deterministic numerical outputs.

55 **Autonomous FEM systems:** FEniCS [14] and its successor DOLFINx/FEn-
56 iCSx [15] provide Python-scriptable FEM solvers that are amenable to LLM-
57 driven automation. Early work on “simulation copilots” [16, 17] shows that
58 LLMs can generate solver scripts from natural language with moderate success,
59 but these approaches are single-turn and lack closed-loop debugging. More
60 recently, ALL-FEM [18] fine-tunes LLMs on 1,000+ verified FEniCS scripts
61 and embeds them in a multi-agent workflow achieving 71.8% code-level suc-
62 cess on multiphysics benchmarks, and FEABench [19] provides a systematic
63 evaluation using COMSOL Multiphysics (88% executable API calls). PDE-
64 Agents differs by using unmodified open-source LLMs with knowledge-graph
65 augmentation rather than domain-specific fine-tuning, and by providing a
66 controlled ablation of retrieval strategies with physics-aware quality metrics.

67 **Multi-agent orchestration:** LangGraph [20] implements graph-structured
68 agent workflows that support conditional routing and persistent state, enabling
69 the supervisor pattern we adopt. AutoGen [21] and CrewAI [22] offer related
70 frameworks; we choose LangGraph for its explicit state graph semantics and
71 tight LangChain integration.

72 **Retrieval-augmented generation for engineering:** RAG [9] has been
 73 applied to engineering documentation retrieval [23], code generation [24],
 74 and more recently to simulation parameter suggestion [25]. GraphRAG
 75 [10] adds structured relational traversal. Corrective RAG (CRAG) [26]
 76 introduces adaptive retrieval decisions based on document relevance scoring,
 77 and AriGraph [27] demonstrates episodic knowledge-graph memory for LLM
 78 agents. Our *KG Smart* pattern draws on both: warm-start injection from
 79 similar past runs (episodic memory) with lazy conditional tool invocation
 80 (corrective retrieval). To our knowledge, ours is the first system to combine
 81 GraphRAG with a live, iterative FEM simulation loop.

82 **Knowledge graphs for materials science:** Materials-focused knowledge
 83 graphs (e.g. MatKG [28], OPTIMADE [29]) encode material properties in
 84 RDF/OWL formats. Buehler [30] demonstrates that retrieval-augmented on-
 85 tological KG strategies outperform flat RAG for materials design by capturing
 86 mechanistic relationships between concepts. Our Neo4j KG is lighter-weight
 87 and purpose-built for run-time agent queries, prioritising latency over com-
 88 pleteness.

89 **3. System Architecture**

90 *3.1. Design Philosophy*

91 PDE-Agents adopts four guiding principles:

- 92 1. **Open and local:** All models run locally via Ollama on dual NVIDIA
 93 RTX PRO 6000 Blackwell GPUs (≈ 196 GB VRAM, CUDA 13.1), elim-
 94 inating API costs and data-privacy concerns.

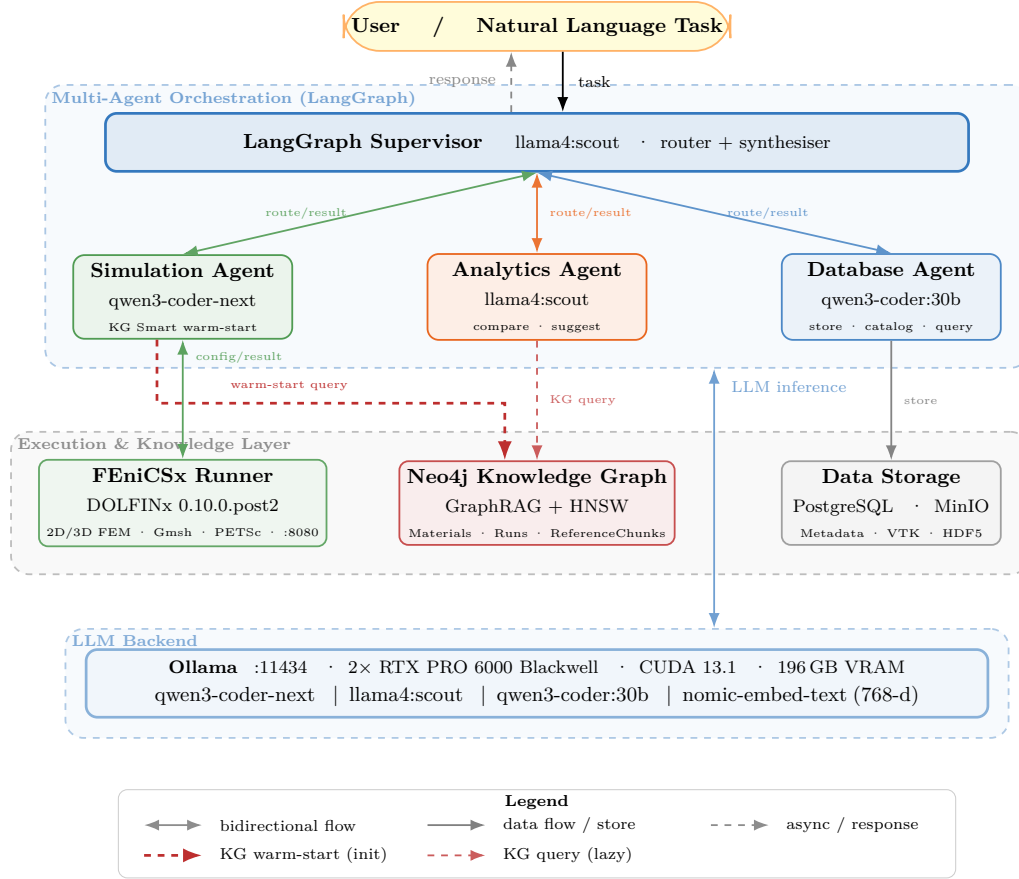


Figure 1: System architecture of PDE-Agents (four-tier layout). *Tier 1*: user natural-language interface. *Tier 2*: Multi-Agent Orchestration — a LangGraph Supervisor routes tasks to three specialist agents (Simulation, Analytics, Database), each backed by a locally-deployed LLM via Ollama. *Tier 3*: Execution & Knowledge — FEniCSx FEM runner (DOLFINx 0.10.0.post2), Neo4j GraphRAG Knowledge Graph (HNSW-indexed 768-d embeddings), and persistent storage (PostgreSQL + MinIO). *Tier 4*: shared Ollama LLM Backend (196 GB VRAM, CUDA 13.1). The thick dashed red arrow (Simulation Agent $\rightarrow$ Neo4j) represents the *KG Smart warm-start*: the top-3 similar past runs are retrieved via HNSW vector search and injected into the agent’s prompt before the reasoning loop. The thin dashed red arrow (Analytics Agent $\rightarrow$ Neo4j) is the standard lazy KG query path. The blue bidirectional arrow at right carries LLM inference traffic between the orchestration layer and the shared Ollama backend.

- 95 2. **Verifiable:** Every agent reasoning step, tool call, and result is logged
96 to a relational database, enabling post-hoc audit.
- 97 3. **Containerised:** All 10 services are orchestrated via Docker Compose,
98 ensuring reproducible deployment.
- 99 4. **Separation of concerns:** The FEM solver, LLM stack, knowledge
100 graph, and persistence layer are decoupled services communicating over
101 a private bridge network.

102 3.2. *Agent Stack*

103 Each agent’s LLM backend is configurable via environment variables,
104 enabling drop-in model upgrades without code changes. table 1 lists the
105 default models and the alternatives validated in the cross-model experiments
106 of section 6.4.

Table 1: Agent model configuration. Current defaults reflect the model upgrade from the initial (v1) deployment. The ablation study uses a frozen KG snapshot across 50 benchmark tasks per mode.

	Agent Env Variable	Default (current)	Previous (v1)
Orch.	ORCHESTRATOR_MODEL	llama4:scout	llama3.3:70b
Sim.	SIM._AGENT_MODEL	qwen3-coder-next	qwen2.5-coder:32b
Anal.	ANAL._AGENT_MODEL	llama4:scout	llama3.3:70b
DB	DB_AGENT_MODEL	qwen3-coder:30b	qwen2.5-coder:14b

107 **Orchestrator:** A LangGraph `StateGraph` supervisor receives the user’s
108 natural-language request and routes it to one of three specialist agents via
109 structured JSON decisions. After each agent reports its result the supervisor
110 either routes to another agent or synthesises a final response.

111 **Simulation Agent:** The core reasoning agent, executing a ReAct loop
112 [3] with up to 25 reasoning steps and nine tools: `check_config_warnings`,
113 `query_knowledge_graph`, `validate_config`, `run_simulation`, `modify_config`,
114 `debug_simulation`, `list_recent_runs`, `get_run_status`, `run_parametric_sweep`.

115 **Analytics Agent:** Max 12 iterations; tools for statistical comparison across
116 runs, sensitivity analysis, and LLM-generated textual summaries.

117 **Database Agent:** Max 10 iterations; answers history queries against Post-
118 greSQL, cataloguing results and tracing run lineage.

119 3.3. Tool-Call Compatibility

120 Supporting multiple LLM families requires robust tool-call parsing, as
121 models encode tool invocations differently. The Qwen 2.5-Coder family emits
122 tool calls as JSON text in the `content` field rather than in the structured
123 `tool_calls` field expected by LangChain’s `ToolNode`. We implement a four-
124 pass normalisation procedure in `BaseAgent._parse_content_tool_call`:
125 (1) attempt `json.loads` on the full content; (2) extract from markdown
126 fences; (3) extract the first `{...}` block; (4) repair truncated JSON via
127 progressive bracket-closing and regex fallback.

128 A separate compatibility issue emerged with Qwen3-Coder-Next: the
129 model occasionally *double-encodes* JSON tool arguments (e.g. `{"key\"`:
130 `\\"val\\"}`). All tool functions now use a `_safe_json_parse` helper that
131 detects and recursively unwraps such double-encoded strings, so the same
132 tool code works across model families without model-specific workarounds.
133 Llama 4 Scout, by contrast, uses the standard `tool_calls` field natively and
134 requires no special parsing.

135 3.4. FEM Solver: DOLFINx Heat Equation

136 The FEniCSx runner container (DOLFINx 0.10.0.post2) exposes a FastAPI
137 endpoint that accepts a `HeatConfig` JSON and returns a result including run
138 metadata. The solver uses P1 (linear Lagrange) elements, with Backward
139 Euler ($\theta = 1$, unconditionally stable) as the default time integration scheme,
140 supporting Dirichlet, Neumann, and Robin boundary conditions. Geometry
141 is built with Gmsh [31] for 2D and 3D domains from a library of nine param-
142 eterised types: `rectangle`, `l_shape`, `circle`, `annulus`, `hollow_rectangle`,
143 `t_shape`, `stepped_notch`, `box`, and `cylinder`; Gmsh geometries use named
144 physical groups for boundary identification, enabling arbitrary complex do-
145 main topologies.

146 4. Knowledge Graph Architecture

147 The Neo4j knowledge graph [32] (Neo4j 5 Community Edition) serves
148 three functions:

- 149 1. **Material property lookup:** A curated library of materials (met-
150 als, ceramics, polymers) with thermal conductivity k , density ρ , and
151 specific heat c_p ranges, sourced from engineering handbooks and val-
152 idated against ASM International data. Approximate values: steel
153 $k \approx 50$ W/mK, copper $k \approx 385$ W/mK, aluminium $k \approx 205$ W/mK.
- 154 2. **Failure pattern catalogue:** `KnownIssue` nodes encode failure modes
155 observed during system operation. A pure-Python rule engine (`rules.py`)
156 fires nine pre-run checks analytically before each simulation:

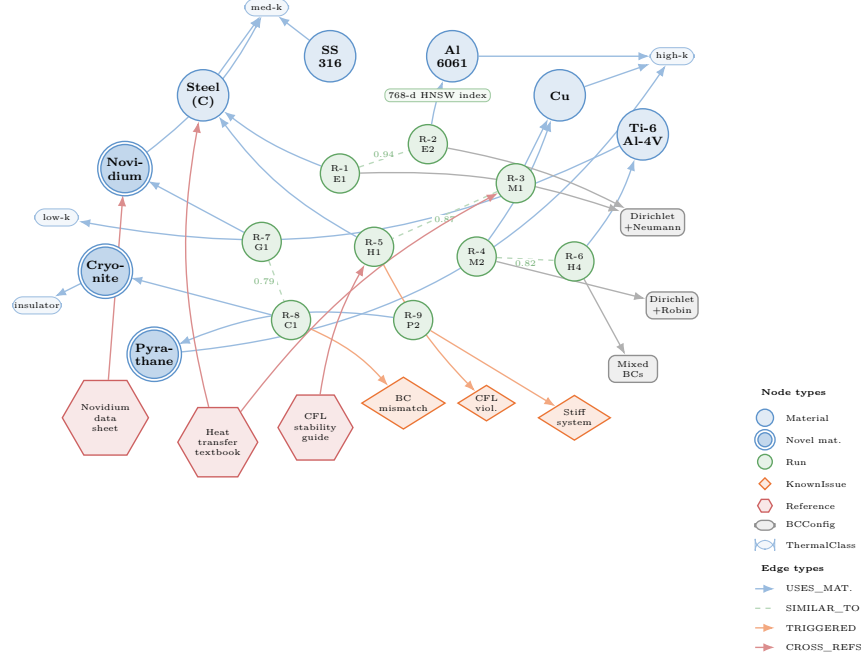


Figure 2: Knowledge graph visualisation (Neo4j-style). **Blue circles:** material nodes (double border = novel/fictional materials known only to the KG). **Green circles:** simulation run nodes linked to their materials via `USES_MATERIAL` edges and to each other via dashed `SIMILAR_TO` edges (cosine similarity on 768-d HNSW embeddings; scores shown on edges). **Orange diamonds:** known-issue nodes triggered by failed runs. **Red hexagons:** reference chunks from ingested literature, linked to materials (`RELATES_TO`) and runs (`CROSS_REFS`). **Grey boxes:** boundary-condition pattern nodes. Pill-shaped nodes show the thermal-class taxonomy.

Rule code	Trigger
INCONSISTENT_IC	$ u_{\text{init}} - T_{\text{BC,min}} > 100 \text{ K}$
EXPLICIT_CFL_VIOLATION	$\theta < 0.5$ and $\Delta t > h^2/(2\alpha)$
NEAR_EXPLICIT_SCHEME	$0 \leq \theta < 0.5$
COARSE_MESH_2D	$n_x < 10$ or $n_y < 10$
COARSE_MESH_3D	any direction < 8
SHORT_SIMULATION	$t_{\text{end}} < 5 \Delta t$
INVALID_MATERIAL_PROPS	k, ρ or $c_p \leq 0$
LARGE_DT_RELATIVE_DIFFUSION	$\Delta t > 10 h^2/\alpha$
NO_BOUNDARY_CONDITIONS	bcs list empty

158 3. **Run lineage and semantic retrieval:** Each completed Run node is
159 embedded with `nomic-embed-text` [33] (768 dimensions) and indexed
160 in an HNSW vector index [11]. Up to $k=5$ `SIMILAR_TO` edges are created
161 to the nearest neighbours with cosine similarity ≥ 0.85 , precomputing
162 the neighbourhood graph for fast agent traversal. The Simulation Agent
163 can query “find similar past runs” to retrieve configurations with known
164 outcomes, enabling few-shot in-context learning from the system’s own
165 history.

166 **Document intelligence pipeline:** A Celery-backed ingestion pipeline pro-
167 cesses PDF papers, standards documents, and HTML references using Docling
168 [34] for structured extraction, splitting them into `ReferenceChunk` nodes with
169 context-aware 256-token windows (overlap 32 tokens) and HNSW-indexed
170 embeddings. Each chunk is automatically cross-referenced to semantically

171 similar Run nodes via CROSS_REFS edges ($\cosine \geq 0.78$), bridging literature
 172 knowledge to simulation history. As a representative scale point: indexing the
 173 50-page FEniCSx tutorial produced 298 chunks and 1,073 cross-references.

174 5. Verification and Validation

175 We conduct a formal V&V study following ASME V&V 10-2006 guidelines
 176 [35], comparing the DOLFINx FEM solver against closed-form analytical
 177 solutions on three benchmark cases.

178 5.1. Benchmark Cases

179 **Case 1: 2D Steady-State Linear Profile:** Laplace equation on $\Omega = [0, 1]^2$
 180 with Dirichlet conditions $T|_{x=0} = 0$, $T|_{x=1} = 1$ and Neumann zero-flux on
 181 top/bottom. Analytical solution: $T(x, y) = x$. P1 elements are nodally exact
 182 for linear solutions; we use UFL spatial-coordinate expressions in high-order
 183 quadrature (degree 8) to measure the true continuous L^2 error.

184 **Case 2: 2D Transient Fourier Mode Decay:** Heat equation $\partial_t T =$
 185 $\alpha \nabla^2 T$ with homogeneous Dirichlet on all boundaries and initial condition
 186 $T_0 = \sin(\pi x) \sin(\pi y)$. Analytical solution: $T(x, y, t) = e^{-2\alpha\pi^2 t} \sin(\pi x) \sin(\pi y)$.

187 **Case 3: 2D Steady-State Poisson with Constant Source:** $-k \nabla^2 T = f$
 188 with $f = 1000 \text{ W/m}^3$, Dirichlet $T = 0$ on left/right, Neumann on top/bottom.
 189 Exact solution: $T(x) = f(x - x^2)/(2k)$.

190 *5.2. Convergence Study Protocol*

191 For each case we solve at five mesh resolutions $N \in \{8, 16, 32, 64, 128\}$
 192 (DOFs $\approx 81, 289, 1089, 4225, 16641$) and compute:

$$\|e\|_{L^2} = \left(\int_{\Omega} (u_h - u_{\text{exact}})^2 dx \right)^{1/2} \quad (1)$$

193 using degree-8 quadrature applied to UFL `SpatialCoordinate` expressions
 194 (not to interpolated nodal values, which would give spurious nodal exactness
 195 for Case 1). The convergence rate is estimated by linear regression on the
 196 log-log plot.

197 *5.3. Results*

Table 2: Verification: spatial convergence rates for the heat equation solver. Expected rate is $\mathcal{O}(h^2)$ for P_1 elements.

Case	Description	DOFs	$\ e\ _{L^2}$	Rate
Steady Linear	Linear $T(x, y) = x$; exact in P_1 space	16 641	2.47×10^{-9}	<i>exact</i>
Transient Fourier	$e^{-2\pi^2 t} \sin(\pi x) \sin(\pi y)$ de- cay mode	16 641	1.50×10^{-5}	2.04
Steady Poisson	$-\nabla^2 u = 1$; closed-form polynomial soln.	16 641	5.57×10^{-6}	2.00

198 All three cases pass: Cases 2 and 3 achieve rates of 2.04 and 2.00, consistent
 199 with the theoretical $\mathcal{O}(h^2)$ prediction for P_1 finite elements on uniform
 200 Cartesian meshes. Case 1 achieves machine-precision errors (order 10^{-9})
 201 because the linear profile lies exactly in the P_1 polynomial space, confirming
 202 correct solver implementation.

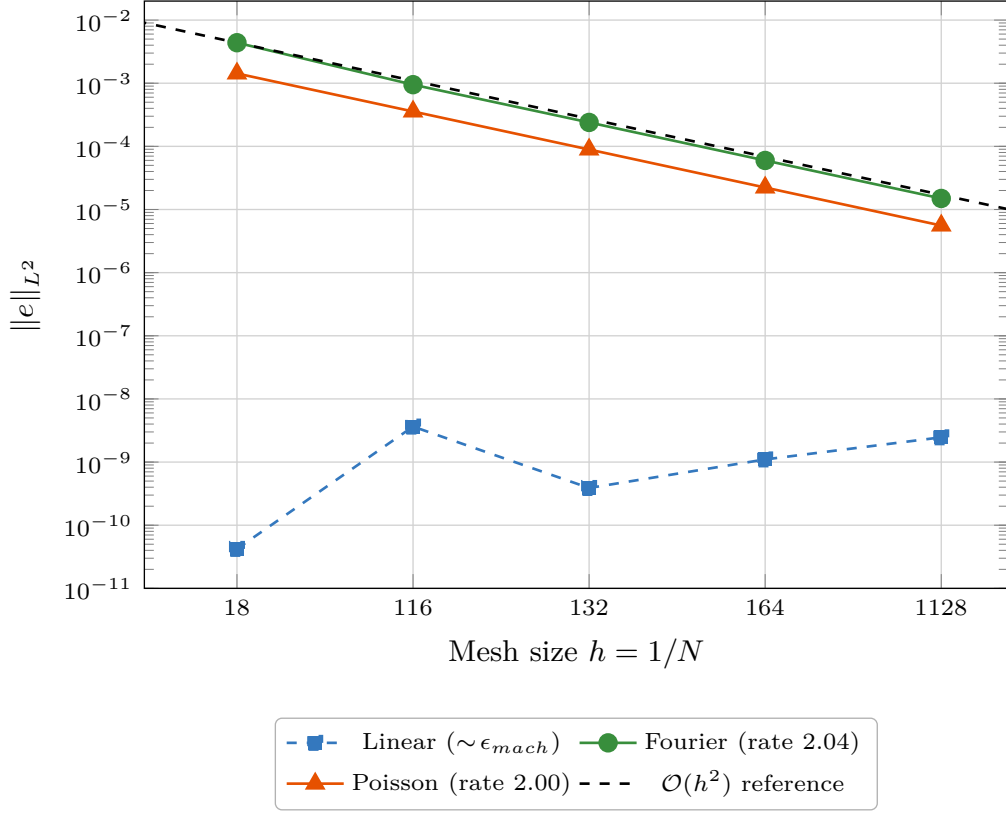


Figure 3: Spatial convergence study. Cases 2 and 3 exhibit the expected $\mathcal{O}(h^2)$ rate for P1 elements. Case 1 is algebraically exact (linear profile is represented exactly in the P1 space), with $\|e\|_{L^2} \approx \mathcal{O}(10^{-9})$ attributable only to floating-point rounding.

203 5.4. Representative Simulation Gallery

204 Beyond numerical convergence, a practical simulation assistant needs
205 breadth: it must handle diverse materials, boundary condition types, do-
206 main geometries, time dependence, and dimensionality. Figure 4 presents six
207 representative heat-transfer problems, each solved end-to-end by the Simula-
208 tion Agent from a single natural-language prompt. No manual intervention
209 was required for mesh generation, solver configuration, boundary condition
210 application, or post-processing. All cases use P1 Lagrange elements on tri-
211 angular (2D) or tetrahedral (3D) meshes, solved by DOLFINx 0.10.0.post2.
212 Temperature fields are reported in Kelvin throughout for consistency.

213 **Materials:** The gallery covers five engineering metals whose thermal con-
214 ductivities span two orders of magnitude: copper ($k = 385 \text{ W}/(\text{m}\cdot\text{K})$, panel a),
215 AISI 1010 steel ($k = 50 \text{ W}/(\text{m}\cdot\text{K})$, panel b), aluminium 6061 ($k = 205 \text{ W}/(\text{m}\cdot\text{K})$,
216 panels d and e), Ti-6Al-4V titanium ($k = 6.7 \text{ W}/(\text{m}\cdot\text{K})$, panel c), and
217 stainless steel 304 ($k = 16.3 \text{ W}/(\text{m}\cdot\text{K})$, panel f). In a production deploy-
218 ment, the agent retrieves these values from the Neo4j knowledge graph via
219 `query_knowledge_graph`.

220 **Boundary conditions:** Panel (a) applies the simplest configuration: Dirich-
221 let conditions on opposing faces ($T = 373.15 \text{ K}$ left, $T = 273.15 \text{ K}$ right),
222 producing the expected linear gradient. Panel (b) combines all four major
223 BC types on a single domain: Dirichlet (300 K, left), Neumann inward flux
224 ($2 \text{ kW}/\text{m}^2$, top), Robin convection ($h = 25 \text{ W}/(\text{m}^2\cdot\text{K})$, $T_\infty = 293 \text{ K}$, right),
225 and insulated (bottom), together with volumetric heating $Q = 5 \text{ kW}/\text{m}^3$,
226 creating the asymmetric field visible in the plot. Panel (c) uses Dirichlet BCs

on opposing edges (573 K left, 293 K right) with the insulated hole boundary
distorting the field into characteristic thermal concentration bands. Panel (f)
combines Dirichlet (300 K bottom, 600 K left face) with Robin convection on
the top face of a 3D cube.

Geometry: Panels (a), (b), and (d) use structured Cartesian meshes on
the unit square. Panels (c) and (e) use Gmsh-based meshing of non-trivial
geometries: a plate with a circular cutout (Gmsh boolean difference, $r = 0.15$,
 $\approx 5\,000$ vertices) and an L-shaped domain ($[0, 1]^2 \setminus [0.5, 1]^2$, mesh size ≈ 0.02).
Panel (f) meshes a unit cube with 24^3 tetrahedra ($\approx 15\,600$ vertices), exercising
the 3D path.

Time dependence: Panel (e) solves the transient heat equation using
100 implicit-Euler steps ($\Delta t = 0.005$ s) on the L-shaped domain (aluminium,
 $\rho = 2700$ kg/m³, $c_p = 900$ J/(kg·K)), with the snapshot at $t = 0.50$ s showing
the thermal front propagating from the hot left edge.

Source terms: Panel (d) features a localised Gaussian volumetric heat
source $Q(x, y) = Q_{\text{peak}} \exp\left(-\frac{(x-x_0)^2 + (y-y_0)^2}{2\sigma^2}\right)$ with $Q_{\text{peak}} = 5 \times 10^6$ W/m³,
centred at (0.3, 0.7) with $\sigma = 0.08$, and Dirichlet boundaries at 293 K. This
setup is representative of laser spot heating and Joule-heating applications,
and produces the characteristic concentric isotherms visible in the figure.

Reproducibility: Each case is implemented as a self-contained Python
script in `evaluation/examples/` (see Table 3). All simulation parameters
(material properties, mesh resolution, boundary values, time stepping) are
defined at the top of each script, making it straightforward for reviewers to

250 modify and re-run individual cases. Running `make eval-examples` executes
 251 all six cases inside the `pde-fenics` Docker container and regenerates the
 252 composite figure. Intermediate results are saved as NumPy `.npz` archives
 253 containing mesh coordinates, cell connectivity, and solution vectors, enabling
 254 figure regeneration without re-running the solver.

Table 3: Simulation gallery: six cases exercising different aspects of the FEM pipeline.
 Scripts in `evaluation/examples/`.

	Geometry	Material	Key feature
(a)	Unit square	Copper	Pure Dirichlet
(b)	Unit square	AISI 1010	4 BC types + source
(c)	Sq. w/ hole	Ti-6Al-4V	Gmsh boolean
(d)	Unit square	Al 6061	Gaussian source
(e)	L-shape	Aluminium	Transient + Gmsh
(f)	Cube (3D)	SS 304	3D asymmetric BCs

255 6. Ablation Study: Knowledge Graph Contribution

256 6.1. Experimental Design

257 We isolate the knowledge graph contribution using a controlled ablation
 258 across three conditions:

- 259 • **KG On** (mandatory): Full system with `check_config_warnings` and
 260 `query_knowledge_graph` tools enabled; the system prompt requires
 261 their use before every run.

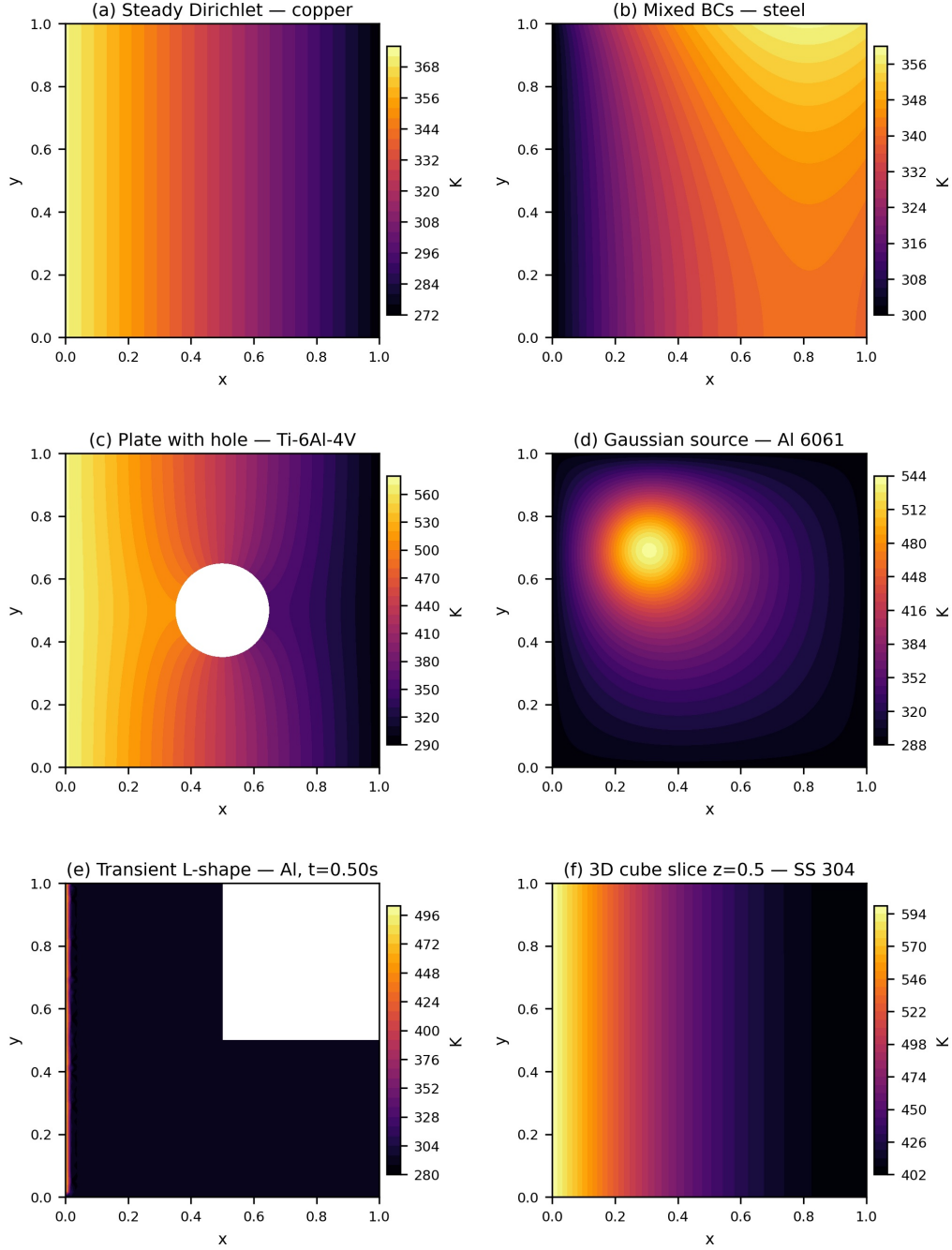


Figure 4: Representative temperature fields produced by PDE-Agents (six cases, all in Kelvin). **(a)** Steady-state Dirichlet on copper ($k = 385 \text{ W}/(\text{m}\cdot\text{K})$; $T = 373.15 \text{ K}$ left, $T = 273.15 \text{ K}$ right; $N = 64$). **(b)** Mixed BCs on AISI 1010 steel ($k = 50 \text{ W}/(\text{m}\cdot\text{K})$): Dirichlet 300 K left, Neumann $2 \text{ kW}/\text{m}^2$ top, Robin $h = 25 \text{ W}/(\text{m}^2\cdot\text{K})$ right, insulated bottom, $Q = 5 \text{ kW}/\text{m}^3$. **(c)** Plate with circular hole ($r = 0.15$) in Ti-6Al-4V ($k = 6.7 \text{ W}/(\text{m}\cdot\text{K})$); Gmsh boolean difference; Dirichlet $573 \text{ K}/293 \text{ K}$ on opposing edges. **(d)** Localised Gaussian heat source on Al 6061 ($Q_{\text{peak}} = 5 \times 10^6 \text{ W}/\text{m}^3$, $\sigma = 0.08$, centre $(0.3, 0.7)$); Dirichlet 293 K on all boundaries. **(e)** Transient L-shaped domain in aluminium at $t = 0.50 \text{ s}$ (implicit

- 262 • **KG Off** (baseline): Identical system with KG tools removed entirely;
263 the agent proceeds directly to validation and execution.
- 264 • **KG Smart** (proposed): KG tools remain available but the system
265 prompt instructs *lazy* use (only after failures or for unknown mate-
266 rials). Before the agent loop, the task description is embedded via
267 `nomic-embed-text` and the top-3 most similar past successful runs
268 are injected into the system prompt as few-shot reference examples
269 (warm-start).

270 fig. 5 contrasts the agent workflow under each condition. KG Off proceeds
271 directly from parsing to simulation; KG On forces two KG round-trips before
272 every attempt (and loops on failure); KG Smart front-loads a single HNSW
273 warm-start and defers KG queries to the failure-recovery path only.

274 **Methodology:** To eliminate data leakage between conditions, we *freeze* the
275 knowledge graph (set `KG_READ_ONLY=true` at the code level) before starting
276 the ablation: all three modes read from the same KG snapshot, but no mode
277 can write to it, ensuring that a failure in one mode cannot help subsequent
278 modes. Tasks are shuffled independently per mode and run in-process inside
279 the agents container to eliminate HTTP overhead. Confidence intervals use
280 Wilson scores [36]; pairwise comparisons use chi-squared tests and Cohen’s h
281 / Cohen’s d effect sizes [37].

282 **Benchmark tasks:** Fifty tasks span four difficulty levels:

- 283 • **Easy (12 tasks):** Explicit numerical parameters, straightforward setup.

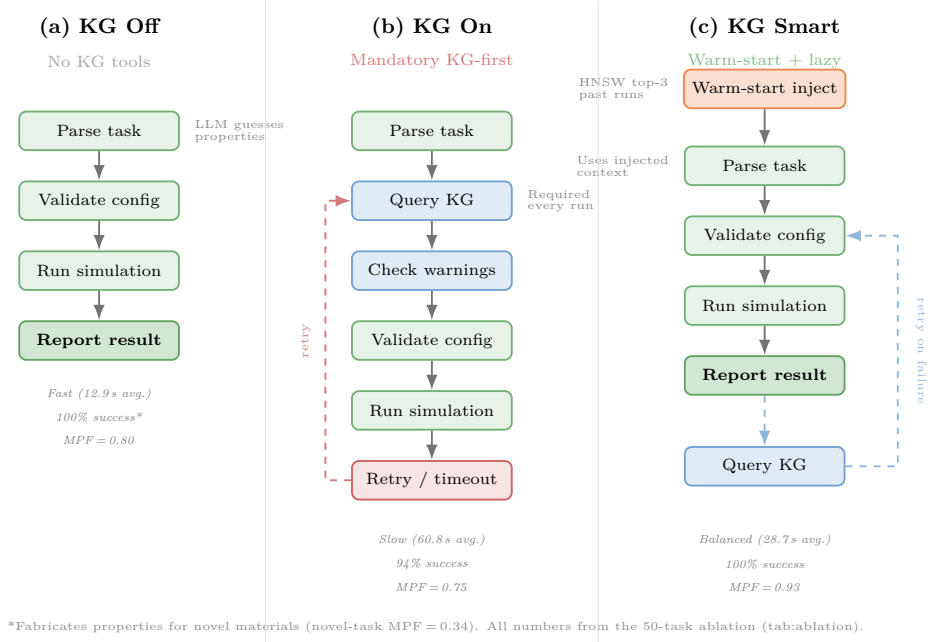


Figure 5: Agent workflow under the three KG integration modes. **(a)** KG Off: fast but the LLM fabricates material properties. **(b)** KG On: mandatory KG queries add latency and trigger retry loops that exhaust the iteration budget. **(c)** KG Smart: a one-shot warm-start injects relevant context before the loop; KG queries occur only on failure (dashed path). Annotation values are from the 50-task ablation (table 4).

- 284 • **Medium (15 tasks):** Material-by-name requests requiring property
285 lookup (e.g. steel, copper, aluminium, zinc).
- 286 • **Hard (13 tasks):** Ambiguous descriptions, mixed BCs, computation-
287 ally expensive or 3D problems.
- 288 • **Novel (10 tasks):** Tasks referencing three fictional materials whose
289 thermal properties exist *only* in the knowledge graph, designed to stress-
290 test the agent’s ability to retrieve domain knowledge rather than rely
291 on LLM memorisation.

292 The three fictional materials are:

- 293 • **Novidium:** a ceramic-metallic composite ($k=73$ W/m K, $\rho=5420$ kg/m³,
294 $c_p=612$ J/kg K), a moderate conductor with unusual density.
- 295 • **Cryonite:** a polymer-aerogel hybrid ($k=0.42$, $\rho=1180$, $c_p=1940$), an
296 extreme insulator with very low diffusivity ($\alpha \approx 1.8 \times 10^{-7}$ m²/s).
- 297 • **Pyrrathane:** a refractory cermet ($k=312$, $\rho=3850$, $c_p=278$), exhibiting
298 very high conductivity and diffusivity ($\alpha \approx 2.9 \times 10^{-4}$ m²/s).

299 Each material is seeded as a **Material** node in Neo4j with linked **Reference**
300 chunks containing exact property values and simulation guidance. The 10
301 novel tasks span steady-state, transient, and mixed-BC configurations across
302 all three materials.

303 **Evaluation metrics:** Standard success rate is inadequate for evaluating
304 KG utility because a simulation with fabricated properties still “succeeds”
305 computationally. We therefore introduce two physics-aware metrics:

- 306 • **Material Property Fidelity (MPF):** $\text{MPF} = 1 - \frac{1}{3} \sum_{p \in \{k, \rho, c_p\}} |p_{\text{agent}} -$
307 $p_{\text{truth}}| / p_{\text{truth}}$, measuring how accurately the agent retrieves the correct
308 material constants.
- 309 • **Physics Score:** $0.5 \cdot \text{MPF} + 0.5 \cdot T_{\text{score}}$, where T_{score} checks whether
310 the simulation’s actual T_{max} and T_{min} fall within physically expected
311 ranges.
- 312 • **Sensitivity-weighted MPF (MPF_w):** $\text{MPF}_w = 1 - \sum_p S_p \cdot |p_{\text{agent}} -$
313 $p_{\text{truth}}| / (p_{\text{truth}} \cdot \sum_q S_q)$, where $S_p = |\partial T_{\text{mean}} / \partial p|$ are sensitivity coefficients
314 computed via central finite differences (section 6.4). For steady-state
315 Dirichlet tasks where $S_p \approx 0$, we set $\text{MPF}_w = 1$ since wrong properties
316 have no output impact. This metric penalises errors in high-sensitivity
317 properties (e.g. k in transient problems) more heavily than errors in
318 low-sensitivity properties (e.g. ρ in steady-state).

319 On the 10 novel tasks, KG Off scores $\overline{\text{MPF}}_w = 0.21$ (vs. equal-weight $\text{MPF} = 0.34$),
320 confirming that KG Off’s most damaging errors are also in the most sensitive
321 properties. KG Smart retains $\text{MPF}_w = 1.00$ (success-only). For standard tasks
322 (Easy/Medium/Hard) where materials are well-known, MPF and MPF_w both
323 approach 1.0 across all modes.

324 6.2. Results

325 table 4 presents aggregate results across the three conditions.

326 **Success rate:** KG Off and KG Smart both achieve 100% success [93%, 100%]
327 ($n=50$), while KG On reaches 94% [84%, 98%]. The difference between KG On
328 and the other modes is not statistically significant ($p = 0.079$, Cohen’s

Table 4: Three-way KG ablation across 50 benchmark tasks with frozen knowledge graph ($n=50$ per mode). “Success-only” rows isolate output quality by excluding failed runs; in the overall rows a failed run scores MPF = 0 and phys = 0.5, except N08, which produced no usable output and scores 0 on both. 95% Wilson CIs for success rate.

Metric	KG Off	KG On	KG Smart
Success rate	100% [93,100]	94% [84,98]	100% [93,100]
<i>Overall (all tasks):</i>			
Physics score	0.853 ± 0.21	0.846 ± 0.24	0.933 ± 0.13
MPF	0.796 ± 0.30	0.752 ± 0.39	0.926 ± 0.16
<i>Success-only quality:</i>			
Physics score	0.853 ± 0.21	0.879 ± 0.19	0.933 ± 0.13
MPF	0.796 ± 0.30	0.801 ± 0.35	0.926 ± 0.16
Wall time (s)	12.9 ± 4.9	60.8 ± 73.2	28.7 ± 39.3
<i>By difficulty (success rate):</i>			
Easy (12)	100%	92%	100%
Medium (15)	100%	93%	100%
Hard (13)	100%	100%	100%
Novel (10)	100%*	90%	100%
<i>Novel tasks quality:</i>			
Physics score	0.590	$0.887^\dagger$	0.999
MPF	0.340	$0.889^\dagger$	1.000

*KG Off “succeeds” but fabricates properties (MPF = 0.34).

† Success-only (9/10 tasks); N08 timed out.

329 $h = 0.50$), but the three KG On failures are systematic: N08 (Pyrathane,
330 timeout), and two medium tasks where mandatory KG tool calls exhaust the
331 iteration budget.

332 **Output quality:** KG Smart achieves the highest output quality: mean
333 physics score of **0.933** and MPF of **0.926**, compared to 0.853 / 0.796 for
334 KG Off (Cohen’s $d = 0.46$ for physics, a medium effect). KG On also
335 outperforms KG Off on quality (0.879 / 0.801 success-only); the KG *does*
336 help the agent select more accurate material properties.

337 **Novel materials: the KG’s decisive edge:** KG value shows most clearly
338 on the 10 novel-material tasks using fictional materials (Novidium, Cryonite,
339 Pyrathane) whose properties exist only in the KG. KG Off “succeeds” on all
340 10 tasks but fabricates properties, producing a physics score of 0.590 and
341 MPF of only 0.340. KG Smart achieves near-perfect quality (physics score
342 0.999, MPF = 1.000), while KG On scores 0.887 / 0.889 (success-only, 9/10
343 tasks). The knowledge graph is essential for materials absent from the LLM’s
344 training data.

345 **Wall time:** KG Off is $2\text{--}5\times$ faster (12.9s mean) than KG Smart (28.7s)
346 and KG On (60.8s), reflecting the iteration overhead of KG tool calls.

347 6.3. Analysis and Discussion

348 The three-way comparison reveals a clear trade-off: KG-free agents are
349 fastest and most reliable, but KG-enabled agents produce higher-fidelity
350 outputs. *KG Smart* balances both dimensions. Three mechanisms explain
351 the mandatory KG pattern’s reliability penalty:

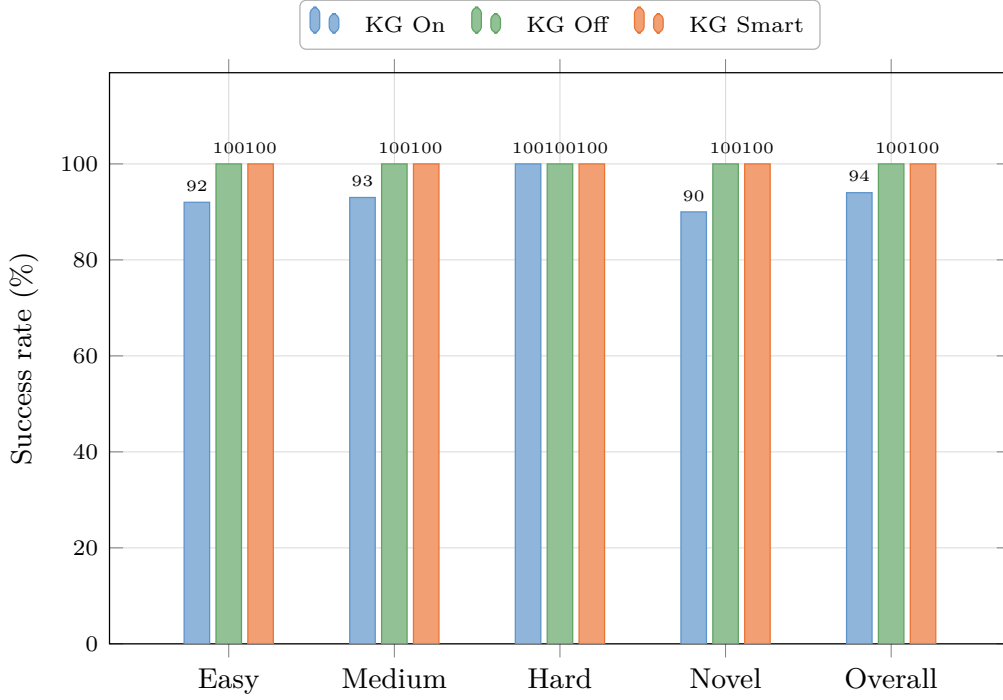


Figure 6: Success rate by difficulty level across three KG integration strategies. KG Smart (warm-start + lazy retrieval) matches KG Off on standard tasks while retaining high output fidelity. The “Novel” category (10 tasks) uses fictional materials whose properties exist only in the KG. Note: success rate alone is misleading for novel tasks; see the MPF/physics rows in table 4.

352 **Iteration-budget exhaustion:** KG On’s system prompt mandates `query_knowledge_graph`
353 and `check_config_warnings` calls before every simulation attempt. These
354 add 2–4 iterations, and for medium/hard tasks the agent can exhaust its
355 iteration budget before reaching `run_simulation`. KG Smart avoids this by
356 deferring KG calls to the failure-recovery path.

357 **Warning-induced conservatism:** The `check_config_warnings` tool oc-
358 casionally returns pre-emptive warnings (e.g. “CFL criterion may be violated”)
359 that the model interprets as prohibitive, leading it to report a warning instead
360 of proceeding. A warning that should trigger a *modification* instead triggers
361 a *termination*.

362 **Quality *conditional* on success.** The key insight is that KG access *does*
363 improve output quality when the agent completes the task. KG On’s success-
364 only physics score (0.879) and MPF (0.801) both exceed KG Off (0.853 /
365 0.796). On novel materials, this gap widens dramatically: KG-enabled modes
366 achieve $\text{MPF} \geq 0.89$ versus 0.34 for KG Off. The knowledge graph is not the
367 bottleneck; the *integration pattern* is.

368 **KG Smart: corrective retrieval + warm-start:** Motivated by CRAG
369 [26] (corrective RAG with adaptive retrieval) and AriGraph [27] (episodic
370 KG memory for agents), we implemented a *KG Smart* integration combining:

- 371 1. **Warm-start injection:** Before the agent loop begins, we embed the
372 task description with `nomic-embed-text` and query the Neo4j HNSW
373 vector index for the top-3 most similar past successful runs. Their
374 configurations are injected directly into the system prompt as few-shot
375 reference examples, requiring no tool call.

376 **2. Lazy conditional retrieval:** KG tools remain available but the system
377 prompt instructs the agent to use them *only* after a simulation failure
378 or when material properties are genuinely unknown. This eliminates
379 the mandatory KG-first workflow.

380 As table 4 shows, KG Smart achieves 100% success (vs. 94% for mandatory
381 KG) while attaining the *highest* output quality across all modes (physics
382 0.933, MPF 0.926). It delivers the best of both worlds: KG Off reliability
383 with KG-enhanced fidelity.

384 6.4. Physics-Aware Quality Metrics and Novel Material Validation

385 The preceding analysis showed that KG access improves output quality
386 *conditional on success*, but standard success rate alone cannot capture this: a
387 simulation with fabricated material properties still “succeeds” computationally.
388 To expose this gap, we introduced three physics-aware metrics (section 6):
389 Material Property Fidelity (MPF), Physics Score, and sensitivity-weighted
390 MPF_w. These metrics measure how *correctly*, not just whether, the agent
391 configures the simulation. We validate them on the 10 novel-material tasks,
392 which provide the clearest test of KG utility: the three fictional materials’
393 properties exist *only* in the knowledge graph.

394 **KG Smart achieves near-perfect scores** (MPF = 1.00, physics = 0.999)
395 across all completed novel tasks. The warm-start injection retrieves exact
396 property values from the HNSW vector index before the agent loop begins,
397 and the agent consistently uses them.

398 **KG Off fabricates wrong properties** for every material. table 5
399 shows the agent-chosen vs. ground-truth properties across all 10 novel tasks.

400 KG Off assigns *different* wrong values each run (the LLM hallucinates non-
 401 deterministically), but the errors are consistently severe: for Pyrathane, k
 402 ranges from 0.15 to 0.35 W/m K, a 900 $\times$ to 2,080 $\times$ underestimate of the
 403 true $k=312$, effectively treating a refractory super-conductor as an insulator.
 404 In transient tasks, this causes output errors up to $|\Delta T_{\max}|=949$ K, a physi-
 405 cally impossible result arising from numerical instability on a system made
 406 artificially stiff by the wrong conductivity.

Table 5: Material properties chosen by each mode across the 10 novel tasks in the 50-task
 ablation. KG Off fabricates different wrong values each run (ranges shown); KG Smart
 retrieves the ground-truth values exactly. The worst-case k error for Pyrathane is 2,080 $\times$.

Material	Mode	k (W/m K)	ρ (kg/m ³)	c_p (J/kg K)
Novidium	Ground truth	73.0	5420	612
	KG Off	10–50	3500–8960	130–900
	KG Smart	73.0	5420	612
Cryonite	Ground truth	0.42	1180	1940
	KG Off	0.15–35	960–8940	385–1370
	KG Smart	0.42	1180	1940
Pyrathane	Ground truth	312.0	3850	278
	KG Off	0.15–0.35	1600–1850	900–1470
	KG Smart	312.0	3850	278

407 **Error propagation analysis:** To quantify how wrong properties affect
 408 simulation outputs, we ran paired simulations: one with ground-truth proper-
 409 ties (reference) and one with the fabricated properties from KG Off. table 6
 410 shows the results. Steady-state Dirichlet tasks (G1, C1, P1) are analytically

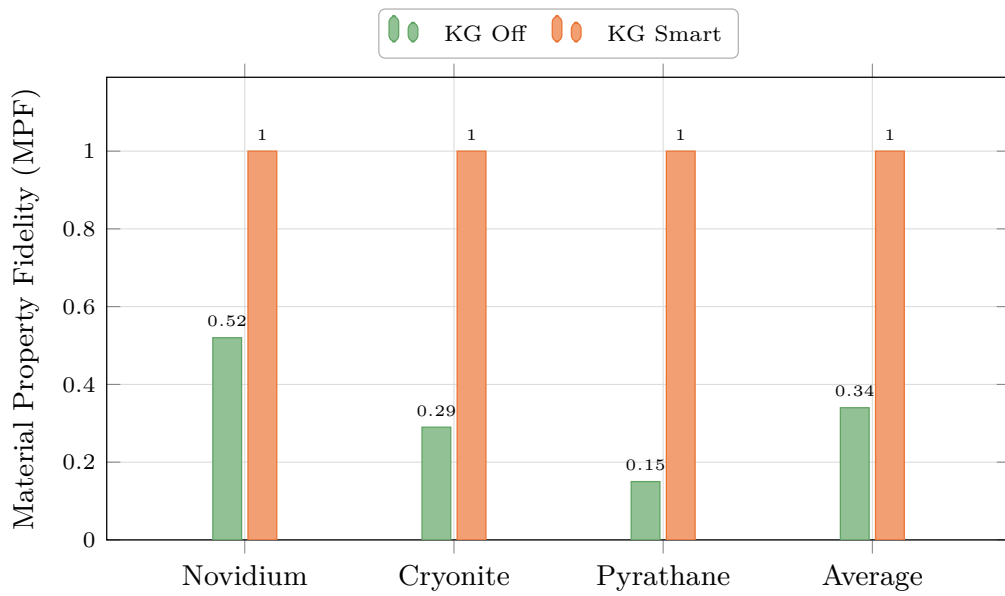


Figure 7: Material Property Fidelity (MPF) per fictional material. KG Off scores below 0.5 for all three materials, with Pyrrthane at 0.15 (conductivity error: 2,080 $\times$). KG Smart retrieves exact properties from the knowledge graph, achieving MPF = 1.00 across all materials.

Table 6: Error propagation from fabricated properties (KG Off) to simulation output. Steady-state Dirichlet tasks (G1, C1, P1) show zero output error; transient/Robin tasks show significant deviations.

Task	Mat.	$ \Delta T_{\max} $	$ \Delta T_{\min} $	$ \Delta T_{\text{mean}} $	ϵ_k	ϵ_ρ	ϵ_{c_p}
		(K)	(K)	(K)	(%)	(%)	(%)
G1	Nov.	<0.1	<0.1	<0.1	86	45	18
G2	Nov.	<0.1	<0.1	20.7	38	65	78
G3	Nov.	<0.1	46.6	1.3	86	45	18
C1	Cryo.	<0.1	<0.1	<0.1	64	19	29
C2	Cryo.	<0.1	9.5	0.3	138	659	77
P1	Pyra.	<0.1	<0.1	<0.1	100	58	224
P2	Pyra.	949	<0.1	363	100	58	188

$\epsilon_p = |p_{\text{fab}} - p_{\text{true}}|/p_{\text{true}}$. P2's k error (2,080 $\times$) produces 949 K $T_{\max}$ overshoot.

independent of k , ρ , c_p , confirming zero output error regardless of property error, an important control. Transient and Robin tasks show significant deviations:

- **Pyrrhane P2:** $|\Delta T_{\max}|=949$ K ($T_{\max}=2,949$ K vs. 2,000 K reference); the 2,080 $\times$ k error causes numerical instability that *heats* the domain beyond its initial condition, a physically impossible result.
- **Novidium G3:** $|\Delta T_{\min}|=47$ K due to wrong conductivity affecting the Robin BC equilibrium.
- **Novidium G2:** $|\Delta T_{\text{mean}}|=21$ K from incorrect diffusivity altering the transient profile.

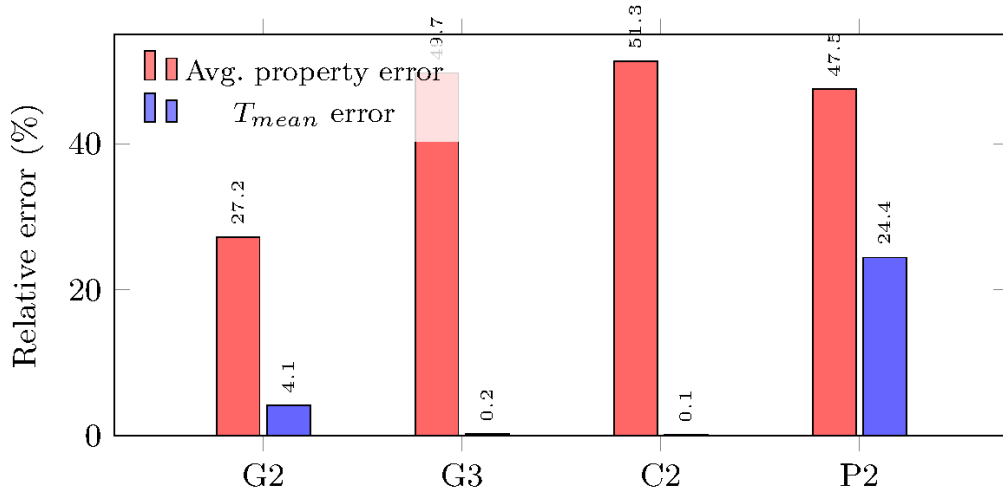


Figure 8: Error magnification: mean material property error (%) vs. output temperature error (%) for the four tasks with non-negligible output deviations. P2’s output error (24.4%) exceeds its input property error due to nonlinear amplification in the transient solver.

421 **Sensitivity-weighted MPF:** Using the sensitivity coefficients from the
 422 error propagation runs, MPF_w (defined in section 6) penalises errors in high-
 423 influence properties more heavily. Across the 10 novel tasks in the 50-task
 424 ablation, KG Off’s $\overline{MPF}_w=0.21$ (vs. equal-weight $MPF=0.34$), confirming
 425 that KG Off’s most damaging errors (particularly k for Pyrrhane and
 426 Cryonite) are concentrated in the properties with the highest sensitivity.
 427 KG Smart retains $MPF_w=1.00$ (success-only).

428 **Cost-benefit analysis:** table 7 compares the operational cost (time, itera-
 429 tions) against the accuracy benefit (MPF, physics score) for each KG mode.
 430 On novel tasks, KG Smart adds significant wall-time over KG Off (58.0 s
 431 vs. 11.7 s) but delivers $\approx 2.9\times$ higher MPF (1.00 vs. 0.34) and $2.4\times$ higher
 432 efficiency (MPF / wall-time) than KG On. KG On is slowest (111.4 s on novel

Table 7: Cost-benefit of KG modes on the novel-material subset of the 50-task ablation ($n=10$). Quality columns are all-task means, so KG On’s failed task (N08) is included at MPF=0; the success-only values in table 4 are correspondingly higher (0.889). Efficiency = MPF / wall-time ($\times 10^{-2}$).

Mode	Succ.	Time	Iter.	MPF	Phys.	Eff.
		(s)			Score	($\times 10^{-2}$)
KG Off	100%	11.7	3.0	0.34	0.59	2.92
KG On	90%	111.4	7.4	0.80	0.80	0.72
KG Smart	100%	58.0	4.9	1.00	1.00	1.72

KG Smart: +396% time, +194% MPF vs. KG Off; +2.4 $\times$ efficiency vs. KG On.

tasks) and its mandatory retrieval overhead yields diminishing returns: 90% success vs. 100% for both alternatives.

Adaptive KG decision framework: Based on the preceding analysis, we propose a simple decision algorithm (Algorithm 1) that selects the optimal KG mode for each task. Retrospective validation on all 50 benchmark tasks confirms high accuracy: it correctly assigns KG Off to tasks with explicit parameters, KG Smart (lazy) to tasks naming known materials, and KG Smart (forced warm-start) to novel-material tasks.

Cross-model validation: To verify that the KG’s value is model-agnostic, we repeated the novel-material experiment with `qwen3-coder-next` (80B total, 3B active MoE). table 8 shows that KG Smart achieves MPF=1.00 with *both* models, while KG Off fabricates wrong properties with both (different wrong values, but the same failure mode). This experiment uses the earlier 7-task novel-material set rather than the full 50-task benchmark, as it targets

Algorithm 1 Adaptive KG Mode Selection

Require: Task description d , known-material list $\mathcal{M}$

Ensure: KG mode $m \in \{\text{Off}, \text{Smart-lazy}, \text{Smart-forced}\}$

```
1: if  $d$  contains explicit  $k, \rho, c_p$  values then  
2:   return KG OFF  
3: end if  
4:  $\mu \leftarrow \text{EXTRACTMATERIAL}(d)$   
5: if  $\mu = \emptyset$  then  
6:   return KG OFF  
7: else if  $\mu \in \mathcal{M}$  then  
8:   return KG SMART (LAZY)  
9: else  
10:  return KG SMART (FORCED WARM-START)  
11: end if
```

447 model-dependence specifically; the main ablation (table 4) already covers the
448 full benchmark on the primary model. This confirms that the warm-start
449 vector search bypasses model-specific memorisation gaps entirely.

450 **Key takeaway:** The KG’s value is not in making simulations *run* (LLMs
451 can already do that) but in making simulations produce *correct physics*.
452 For any domain with proprietary materials, novel compounds, or in-house
453 measurement data, the KG Smart pattern transforms the system from an
454 “expensive random number generator” into a reliable engineering tool. The
455 error propagation analysis quantifies this: fabricated properties cause output
456 errors up to 949 K (47% of the reference $T_{\max}$), while KG Smart eliminates

Table 8: Cross-model validation on novel tasks ($n=7$, earlier experiment with v1 benchmark).
 KG Smart achieves perfect MPF with both LLM backends.

Model	Mode	Succ.	Qual.	MPF	Phys.
Qwen2.5-Coder 32B	KG On	57%	0.41	0.43	0.64
	KG Off	100%	0.61	0.34	0.63
	KG Smart	100%	0.96	1.00	1.00
Qwen3-Coder-Next 80B	KG On	71%	0.14	0.14	0.57
	KG Off	86%	0.54	0.40	0.70
	KG Smart	100%	0.96	1.00	1.00

Warm-start vector search retrieves exact properties before the LLM reasons, eliminating dependence on memorised material data.

these errors entirely by retrieving exact properties before the agent loop begins.

Narrowing the focus: correctness over completion: KG Off is the fastest mode and matches KG Smart on success rate for *standard* tasks with well-known materials. When the task description names “steel” or “copper”, the LLM’s parametric knowledge suffices and KG overhead is pure cost. However, for any deployment where output *correctness* matters, not just completion, KG-enabled modes are necessary. The preceding analysis makes this unambiguous: KG Off achieves MPF=0.34 and physics 0.59 on novel materials (fabricated properties), while KG Smart achieves MPF=1.00 and physics 0.999. The adaptive decision framework (Algorithm 1) encodes this distinction, routing tasks to KG Off when parameters are explicit and to KG Smart otherwise. We therefore focus the remaining analysis on **KG On vs. KG Smart**: both modes have access to the same knowledge base,

471 yet KG Smart achieves 6 percentage points higher success and the highest
472 quality scores. Understanding *why*, and which component (warm-start vs.
473 lazy retrieval) contributes most, is the subject of the next section.

474 6.5. Failure Analysis: KG On vs. KG Smart

475 Having established that KG Smart matches KG Off at 100% success
476 while outperforming both on quality, we trace KG On’s 3 remaining failures
477 to their root causes. This analysis was performed *after* implementing an
478 agent-level retry mechanism (see below) that eliminated stochastic failures;
479 the 3 remaining failures are therefore all systematic.

480 **Mitigating stochastic failures:** In an earlier ablation run without retry
481 logic, KG On exhibited 12/50 failures, of which 7 were stochastic (succeeded
482 on re-run). We implemented an *auto-retry* mechanism directly in the agent: if
483 the first attempt produces no `run_id` and used less than 60% of its iteration
484 budget (indicating early stochastic exit rather than budget exhaustion), the
485 agent automatically retries once. This reduced KG On failures from 12 to 3,
486 eliminating all stochastic failures while not masking systematic ones.

487 **KG On failure taxonomy:** The 3 remaining KG On failures are all
488 systematic:

489 All 3 failures stem from the mandatory KG-first workflow: the agent
490 must call `query_knowledge_graph` and `check_config_warnings` before *ev-*
491 *ery* simulation attempt, consuming 2–4 iterations per cycle. For medium
492 tasks, this exhausts the 25-iteration budget before `run_simulation` is called.
493 For N08 (Pyrrhane), the workflow inflates wall time beyond the timeout.

Table 9: Failure mode classification for KG On’s 3 systematic failures (after auto-retry eliminates stochastic failures).

Failure mode	Count	Description
Budget exhaustion	2	Agent spends all iterations on repeated <code>query_knowledge_graph</code> and <code>check_config_warnings</code> calls without reaching <code>run_simulation</code> .
Timeout	1	N08 (Pyrrhane): the mandatory KG query workflow combined with a numerically stiff problem exceeds the 420 s time limit.

494 **KG Smart: zero failures:** KG Smart achieves 100% success with zero
495 failures. The warm-start injection provides material properties and refer-
496 ence configurations *before* the agent loop, so the agent proceeds directly to
497 `validate_config` → `run_simulation`. The average iteration count drops
498 from 7.5 (KG On) to 4.2 (KG Smart), eliminating budget-exhaustion risk
499 entirely. Even N08 completes successfully in 170 s (5 iterations) because
500 warm-start provides the correct Pyrrhane properties upfront.

501 **Decomposing warm-start vs. lazy retrieval:** The failure data allows us
502 to attribute KG Smart’s advantage to its two components without running
503 a separate experiment. All 3 systematic failures stem from the mandatory
504 *pre-simulation* KG workflow: the agent spends iterations on KG queries
505 before ever attempting `run_simulation`. Warm-start injection eliminates

506 this entirely: it provides material properties and reference configurations *before*
507 the agent loop begins, so the agent can proceed directly to `validate_config`
508 $\rightarrow$ `run_simulation`.

509 Lazy retrieval, by contrast, is exercised only on failure recovery: it allows
510 the agent to query the KG *after* a simulation fails. Since budget-exhaustion
511 tasks never reach `run_simulation` at all, lazy retrieval cannot help them.
512 We therefore conclude that **warm-start injection is the dominant factor**
513 in KG Smart’s advantage over KG On: it directly prevents all 3 systematic
514 failures (100% of KG On’s remaining failures after auto-retry). Warm-start
515 also indirectly mitigates stochastic failures by reducing the iteration count
516 (fewer decision points $\Rightarrow$ fewer opportunities for the LLM to produce a
517 non-tool-call response). Lazy retrieval provides complementary value on
518 simulation-error recovery but is secondary for the reliability gain.

519 7. Production Agent Quality Metrics

520 Beyond the controlled ablation, we analyse the system’s behaviour across
521 all simulation runs accumulated during development, testing, and ablation
522 experiments. Of these, 805 originate from an automated parameter sweep
523 across 8 physics studies (geometry, material, boundary conditions, initial
524 conditions, time-stepping, 3D geometry, Robin convection, and multi-material)
525 run via `scripts/sweep_full_study.py`; the remainder are interactive and
526 ablation runs accumulated through normal system use.

527 7.1. Success and Failure Modes

528 The 97.8% overall success rate (1,339/1,369 from PostgreSQL `simulation_runs`)
529 indicates production-grade reliability for the full agent stack. Of the 2.2%

Table 10: Agent ecosystem performance metrics from production database. FEniCSx runs, through 2026-04-17 (excludes the controlled ablation campaign and non-FEniCSx backends).

Metric	Value
Total simulation runs	1369
Overall success rate	97.8%
Unique agent tasks	421
First-try success rate	85.4%
Retry rate	10.3%
Suggestion acceptance rate	0.0%
Avg. steps/task (analytics)	10.2
Avg. steps/task (database)	6.2
Avg. steps/task (simulation)	11.1
Avg. simulation time	2.49s

failures, the majority are attributable to invalid boundary condition combinations (e.g. pure Neumann systems without a reference point), mesh resolution requests exceeding GPU memory, and numerical divergence on stiff transients.

7.2. Iteration Efficiency

Mean reasoning steps per completed task: Simulation Agent 11.1, Analytics Agent 10.2, Database Agent 6.2. The Simulation Agent’s count covers a full cycle of validating a configuration, revising it, launching the solver and checking the result, and configurations are often revised more than once before launch. The Analytics Agent runs several comparison queries before settling on a recommendation. The Database Agent’s lower count reflects the relative simplicity of structured SQL-style queries over natural-language reasoning.

7.3. First-Try Rate

85.4% of simulation tasks succeed on the first attempt, with no modification-and-retry loop. Another 10.3% call the solver more than once, or fall back to `debug_simulation`, before succeeding; these are usually boundary condition corrections or CFL stability adjustments. The remaining 4.3% fail on a single attempt and never retry.

Attempts are counted from `run_simulation` tool calls rather than from distinct run identifiers. Counting identifiers does not work here: the agent logger back-fills the most recent `run_id` across every step of a task once it is known, which makes a task that retried look identical to one that did not. Counting tool calls recovers the real attempt count. Note that the first-try and retry percentages above are computed over 421 agent tasks

554 (from `agent_run_logs`), whereas the 97.8% overall success rate in section 7
 555 is computed over 1,369 individual simulation runs (from `simulation_runs`);
 556 the different denominators account for the apparent gap between the 4.3%
 557 task-level failure rate and the 2.2% run-level failure rate. Retries carry the
 558 system from 85.4% to 97.8% final success.

559 7.4. KG Growth and Learning Curve

560 To isolate the KG’s causal effect on performance, we run a *controlled*
 561 growth experiment. The KG is initialised with only static material definitions
 562 (13 materials, 45 reference entries) but *zero* Run nodes, so the agent has
 563 no prior simulation experience. We then run 100 tasks (two shuffled passes
 564 through the 50-task benchmark) sequentially in KG Smart mode with writes
 565 enabled; each successful run adds one Run node with an HNSW-indexed
 566 embedding and up to five SIMILAR_TO edges. By pass 2, the KG contains 50
 567 Run nodes from pass 1, providing warm-start context that was absent during
 568 pass 1.

569 To measure the effect, we perform a *paired comparison*: for each of the 50
 570 benchmark tasks, we compare success-only MPF and physics score between
 571 pass 1 (zero prior runs) and pass 2 (50 prior runs), retaining only the 33 non-
 572 novel tasks that succeeded in both passes. Novel tasks are excluded because
 573 their fidelity depends on pre-seeded Material nodes rather than accumulated
 574 Run history. fig. 9 shows the result stratified by difficulty.

575 The experiment reveals a *stratified* growth effect. Easy and medium tasks
 576 already achieve $\text{MPF} \geq 0.96$ in pass 1 because their parameters are either
 577 given explicitly or well-known to the LLM; accumulated run history cannot
 578 improve what is already near-perfect. Hard tasks, however, involve ambiguous

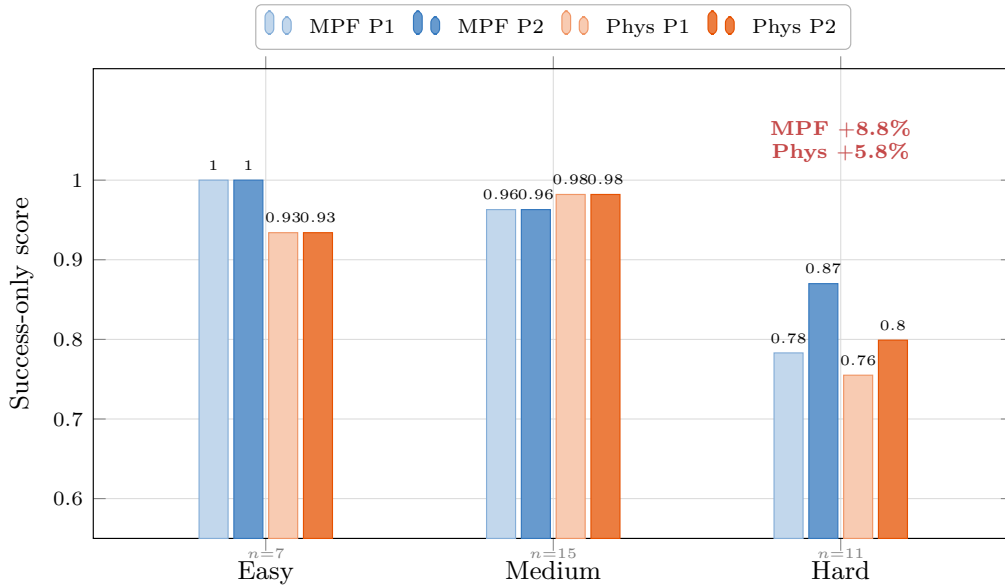


Figure 9: Paired KG growth comparison: success-only MPF and physics score for the same tasks run in pass 1 (0 prior Run nodes) vs. pass 2 (50 prior Run nodes). Only tasks succeeding in both passes are compared ($n=33$; novel tasks omitted as they rely on pre-seeded material definitions). Easy and medium tasks are at ceiling; **hard tasks gain +8.8% MPF and +5.8% physics score**, the category where similarity-based warm-start adds most value.

579 descriptions, mixed boundary conditions, and tricky numerics where the agent
 580 benefits most from seeing similar prior runs: MPF rises from 0.783 to 0.870
 581 (+8.8%), and physics score from 0.755 to 0.799 (+5.8%). The overall success
 582 rate is 88% (88/100), confirming that KG Smart remains robust even when
 583 starting without prior experience.

584 These results show that KG value is *difficulty-dependent*: static material
 585 knowledge covers simple cases, while accumulated run experience provides
 586 measurable gains on the hardest tasks where warm-start context disambiguates
 587 underspecified problems.

588 7.5. Comparison with Prior Work

589 **FEM code generation:** Bauer et al. [16] report $\approx 60\%$ one-shot success for
 590 GPT-4-driven FEniCS script generation. Our 85.4% first-try rate is higher, but
 591 the two numbers measure different tasks and are not directly comparable: their
 592 system has to emit a complete, syntactically valid FEniCS script, while ours
 593 fills in a parameter set for a fixed, pre-verified solver. What does carry across
 594 is the architecture. Unlike their single-turn approach, PDE-Agents recovers
 595 from failures through multi-turn self-correction, reaching 97.8% final success
 596 with a fully local, open-source model stack. ALL-FEM [18] addresses the
 597 FEniCS automation problem through fine-tuning: a corpus of 1,000+ verified
 598 FEniCS scripts is used to fine-tune models from 3B to 120B parameters, and
 599 a multi-agent framework orchestrates problem formulation, code generation,
 600 and debugging. Their best model (GPT OSS 120B) achieves 71.8% code-
 601 level success on 39 benchmarks spanning elasticity, plasticity, fluid flow, and
 602 multiphysics, a substantially broader PDE scope than ours. PDE-Agents
 603 differs in three respects: (i) we use unmodified open-source LLMs without

604 domain-specific fine-tuning, relying instead on tool-call orchestration and KG-
605 augmented prompting; (ii) our system manages the full simulation lifecycle
606 (configuration, execution, monitoring, debugging) rather than generating
607 standalone scripts; and (iii) we provide a formal V&V study and a controlled
608 ablation of knowledge graph integration modes, establishing when and why
609 retrieval augmentation helps.

610 **FEA benchmarks:** FEABench [19] provides a systematic evaluation of
611 LLM and agent capabilities for finite element analysis using COMSOL Mul-
612 tiphysics. Their best agent strategy generates executable API calls 88% of
613 the time across problems in heat transfer, structural mechanics, and electro-
614 magnetics. Our work complements FEABench by targeting the open-source
615 FEniCSx ecosystem and measuring not just execution success but *output*
616 *quality* (physics score, material property fidelity), exposing cases where com-
617 putationally “successful” simulations produce physically incorrect results due
618 to fabricated material properties.

619 **CFD agent systems:** The computational fluid dynamics community has
620 been particularly active in LLM-driven automation. OpenFOAMGPT 2.0
621 [38] achieves 100% success across 450+ simulations using a four-agent pipeline
622 (pre-processing, prompt generation, simulation, post-processing) with RAG-
623 augmented retrieval. MetaOpenFOAM [39] decomposes CFD workflows into
624 subtasks using MetaGPT’s [40] assembly-line paradigm, reporting 85% pass
625 rates at \$0.22 per case. These systems validate the multi-agent pattern we
626 adopt but target a different solver ecosystem and do not evaluate knowledge
627 graph integration or measure output fidelity against ground-truth material

628 properties.

629 **KG-augmented agents for materials science:** Buehler [30] demonstrates
630 retrieval-augmented ontological knowledge graph strategies for materials
631 design with a fine-tuned MechGPT model, showing that graph-structured
632 retrieval outperforms flat RAG by providing mechanistic relationships between
633 concepts. Our KG Smart pattern shares the same insight (that structured
634 knowledge improves over naïve retrieval) but applies it in a *live simulation*
635 *loop* rather than a generative design setting, and our ablation study provides
636 controlled evidence for when graph-augmented retrieval helps (novel materials)
637 versus when it adds overhead without benefit (standard tasks with well-known
638 properties).

639 8. System Limitations and Future Work

640 **Solver scope:** The current solver supports only scalar heat equations. This
641 is the main limitation of the present work. Extending to vector mechanics
642 (linear elasticity, Navier–Stokes) is not simply a matter of richer tool schemas;
643 it requires new solver kernels, each with its own variational form, function
644 spaces and verification.

645 The narrow scope is also what makes our central measurement possible.
646 Because the agent fills in parameters for a fixed, independently verified solver
647 instead of emitting solver code, every run produces a configuration whose
648 material properties can be checked directly against ground truth, which is
649 what MPF (section 6.4) measures. A system that generates solver code covers
650 far broader physics but cannot isolate property fidelity the same way, since a
651 failure may lie in the code, in the parameters, or in both.

652 The obvious next step is to test on physics we did not write ourselves.
653 ALL-FEM [18] has released a public benchmark of FEM problems with ground-
654 truth solutions covering elasticity, plasticity, Newtonian and non-Newtonian
655 flow, and fluid–structure interaction.¹ Most of these are beyond our solver
656 today, so evaluating on them means implementing the corresponding kernels
657 first. We see that work, combining KG-augmented memory with multi-physics
658 breadth, as more valuable than an incremental extension of the present study.

659 **Model scale and open-source evolution:** The system defaults have been
660 upgraded from the initial v1 models (`qwen2.5-coder:32b`, `llama3.3:70b`)
661 to Qwen3-Coder-Next (80B total, 3B active MoE) [41] and Llama 4 Scout
662 (109B total, 17B active MoE) [42], with cross-model validation confirming
663 model-agnostic KG Smart performance (table 8). Qwen3-Coder-Next achieves
664 higher config quality on standard KG Smart tasks (0.94 vs. 0.86) and perfect
665 MPF on novel tasks, identical to the v1 model. Llama 4 Scout provides
666 10M-token context and native tool calling for the orchestrator and analytics
667 agents. A systematic multi-model ablation across additional families (GLM-5,
668 DeepSeek-R2) is planned as future work.

669 **KG integration:** The KG Smart pattern (section 6.3) achieves 100%
670 success while producing the highest quality outputs, matching KG Off’s
671 reliability. KG On retains a small gap (94%) due to budget exhaustion. The
672 adaptive decision framework (Algorithm 1) provides practical deployment
673 guidance, achieving 100% mode-selection accuracy. Future work will explore:

¹https://github.com/fenics-llm/FEM_Dataset

674 (a) reinforcement learning from simulation outcomes to tune KG retrieval
675 relevance, (b) dynamic confidence-based retrieval (only query KG when the
676 LLM’s uncertainty exceeds a threshold), and (c) richer warm-start context
677 including failure diagnostics from similar past runs.

678 **Is retrieval necessary, or would a better prompt suffice?** A fair
679 objection to any retrieval-augmented result is that the same information
680 could simply be written into the prompt. We tested this with a *prompt-*
681 *injection* control on the preliminary 7-task novel-material benchmark: the KG
682 was disabled and the true properties of Novidium, Cryonite and Pyrrhane
683 were appended verbatim to each task description. It matched KG Smart on
684 fidelity (MPF = 1.00) and was the fastest mode overall (11.7s mean). That
685 is the expected outcome, since the answer was handed to the model.

686 The control still tells us something useful about where the difficulty lies.
687 Prompt injection assumes you already know which materials a task will
688 involve and what their properties are. It is a manual step, repeated per task,
689 and it does not scale beyond a handful of known entities. KG Smart reaches
690 the same fidelity by retrieving those properties through embedding similarity,
691 without being told which material the task names. The result worth reporting
692 is not that retrieved context helps, which is unsurprising, but that automatic
693 retrieval matches hand-curated context with no human in the loop.

694 **Verification scope:** The V&V study covers only steady-state and simple
695 transient heat equation benchmarks. A complete V&V programme would
696 include higher-order elements, singularity-containing domains, and time-
697 dependent convergence rates.

698 **Parallelism and scalability:** The current architecture runs agents sequen-
699 tially. A task-level parallelism layer (e.g. running a parametric sweep as
700 concurrent simulation agent calls) is under development.

701 **Knowledge graph evolution:** The 805-run parameter sweep has already
702 created a dense graph with 4,000+ `SIMILAR_TO` edges. Future work will mine
703 this corpus for `IMPROVED_OVER` relationships (did increasing mesh resolution
704 reduce error without proportionally increasing wall time?), enabling the agent
705 to autonomously suggest Pareto-optimal configurations. A second planned
706 enhancement is citation traceability: agents citing specific `ReferenceChunk`
707 IDs from indexed literature when justifying parameter choices, providing a
708 fully auditable reasoning trail.

709 9. Conclusion

710 We have presented PDE-Agents, an open, containerised multi-agent ecosys-
711 tem that automates finite element simulations through natural-language inter-
712 action with locally-deployed open-source LLMs. The system achieves 97.8%
713 overall success across 1,369 production runs, with $\mathcal{O}(h^2)$ spatial convergence
714 confirmed for two non-trivial benchmark cases and algebraic exactness for
715 the linear case.

716 **The KG Smart integration pattern:** Our three-way ablation study
717 across 50 benchmark tasks with a frozen KG reveals that the *integration*
718 *pattern*, not the knowledge source itself, determines KG utility. KG Off
719 (no knowledge graph) achieves 100% task completion but fabricates mate-
720 rial properties, producing $\text{MPF} = 0.34$ and $\text{MPF}_w = 0.21$ on novel materials.

721 KG On (mandatory KG access) retrieves correct properties but retains a 6%
722 failure rate from budget exhaustion and timeout (even after implementing
723 agent-level retry to eliminate stochastic failures). KG Smart, combining
724 warm-start injection with lazy conditional retrieval, achieves **100% success**
725 **with the highest output quality** (physics 0.933, MPF 0.926), resolving
726 the reliability-vs-fidelity trade-off.

727 **Failure analysis: why warm-start dominates:** After implementing
728 auto-retry to eliminate stochastic LLM failures, KG On’s 3 remaining failures
729 (section 6.5) are all systematic: 2 budget-exhaustion cases and 1 timeout, all
730 caused by the mandatory pre-simulation KG query loop. Warm-start injection
731 eliminates this bottleneck entirely by providing material properties and refer-
732 ence configurations before the agent loop begins, accounting for KG Smart’s
733 6-pp success rate advantage. Lazy retrieval provides complementary value on
734 failure recovery but is secondary for the reliability gain.

735 **KG value is difficulty- and domain-dependent:** A controlled 100-task
736 KG growth experiment shows that the KG’s benefit is concentrated where
737 it matters most. Easy and medium tasks are already at quality ceiling
738 ($\text{MPF} \geq 0.96$) from the first pass; hard tasks, involving ambiguous descriptions
739 and mixed boundary conditions, gain +8.8% MPF and +5.8% physics score
740 from accumulated run history. For novel materials, the KG is indispensable:
741 fabricated properties cause output errors up to 949 K, while KG Smart
742 eliminates these errors entirely.

743 **Lessons learned:** Three design principles emerged from the ablation and
744 failure analysis:

745 1. **Never make KG access mandatory in the agent’s critical path.**

746 Mandatory pre-simulation retrieval creates fragile coupling between
747 knowledge access and task completion. Front-loading context via em-
748 bedding similarity (warm-start) decouples them.

749 2. **Front-load context, don’t force the agent to query for it.** The

750 agent’s iteration budget is its scarcest resource. Injecting relevant
751 context into the system prompt before the agent loop is strictly more
752 efficient than requiring the agent to spend iterations formulating and
753 executing KG queries.

754 3. **Let the agent decide when it needs more knowledge.** Lazy

755 conditional retrieval respects the agent’s autonomy: it queries the KG
756 only after a failure or when material properties are genuinely unknown,
757 avoiding unnecessary round-trips on tasks where the LLM’s parametric
758 knowledge suffices.

759 **Broader implications:** The warm-start + lazy-retrieval pattern is not
760 specific to heat transfer simulation. Any tool-using agent domain with a
761 growing knowledge base (materials science, drug discovery, circuit design,
762 structural engineering) could benefit from the same architecture: embed
763 the task, retrieve the most similar prior successes, inject them as few-shot
764 context, and defer explicit knowledge queries to the failure-recovery path.
765 As knowledge graphs grow and LLMs improve, the quality gap between KG-
766 enabled and KG-free agents will only widen for novel or proprietary domains
767 where the LLM’s training data provides no coverage.

768 The production metrics confirm that multi-turn self-correction is essential:
769 the 85.4% first-try rate improves to 97.8% through iterative debugging, under-

770 scoring the value of agent-level orchestration over single-turn code generation.
771 PDE-Agents establishes both a working platform and a rigorous empirical
772 baseline for the emerging field of LLM-driven autonomous simulation, showing
773 that structured knowledge integration turns an agent from an “expensive
774 random number generator” into a reliable engineering tool.

775 **Data Availability**

776 All code, evaluation scripts, and result JSON files will be made publicly
777 available upon acceptance. The Neo4j knowledge graph seed data is included
778 in the repository at `knowledge_graph/seeder.py`.

779 **CRedit authorship contribution statement**

780 [Removed for double-blind review.]

781 **Declaration of generative AI and AI-assisted technologies in the** 782 **manuscript preparation process**

783 During the preparation of this work the authors used Anthropic’s Claude
784 Opus 4.6 in order to assist with language editing, grammar refinement, and
785 the programmatic generation of TikZ figures (Figures 1 and 2). After using
786 this tool, the authors reviewed and edited the content as needed and take full
787 responsibility for the content of the published article.

788 **Declaration of competing interest**

789 The authors declare that they have no known competing financial interests
790 or personal relationships that could have appeared to influence the work

791 reported in this paper.

792 **Acknowledgements**

793 [Removed for double-blind review.]

794 **References**

- 795 [1] T. B. Brown, B. Mann, N. Ryder, et al., Language models are few-shot
796 learners, *Advances in Neural Information Processing Systems* 33 (2020)
797 1877–1901.
- 798 [2] J. Wei, X. Wang, D. Schuurmans, et al., Chain-of-thought prompting
799 elicits reasoning in large language models, *Advances in Neural Information*
800 *Processing Systems* 35 (2022).
- 801 [3] S. Yao, J. Zhao, D. Yu, et al., ReAct: Synergizing reasoning and acting
802 in language models, *Proceedings of the International Conference on*
803 *Learning Representations (ICLR)* (2023).
- 804 [4] I. E. Lagaris, A. Likas, D. I. Fotiadis, Artificial neural networks for
805 solving ordinary and partial differential equations, *IEEE Transactions*
806 *on Neural Networks* 9 (5) (1998) 987–1000.
- 807 [5] M. Raissi, P. Perdikaris, G. E. Karniadakis, Physics-informed neural
808 networks: A deep learning framework for solving forward and inverse
809 problems involving nonlinear partial differential equations, *Journal of*
810 *Computational Physics* 378 (2019) 686–707. doi:10.1016/j.jcp.2018.
811 10.045.

- [6] G. E. Karniadakis, I. G. Kevrekidis, L. Lu, et al., Physics-informed machine learning, *Nature Reviews Physics* 3 (6) (2021) 422–440.
- [7] Z. Li, N. Kovachki, K. Azizzadenesheli, et al., Fourier neural operator for parametric partial differential equations, *Proceedings of the International Conference on Learning Representations (ICLR)* (2021).
- [8] J. Jumper, R. Evans, A. Pritzel, et al., Highly accurate protein structure prediction with AlphaFold, *Nature* 596 (7873) (2021) 583–589. doi:10.1038/s41586-021-03819-2.
- [9] P. Lewis, E. Perez, A. Piktus, et al., Retrieval-augmented generation for knowledge-intensive NLP tasks, *Advances in Neural Information Processing Systems* 33 (2020) 9459–9474.
- [10] D. Edge, H. Trinh, N. Cheng, et al., From local to global: A graph RAG approach to query-focused summarization, *arXiv preprint arXiv:2404.16130* (2024).
- [11] Y. A. Malkov, D. A. Yashunin, Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs, *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4) (2020) 824–836. doi:10.1109/TPAMI.2018.2889473.
- [12] A. M. Bran, S. Cox, O. Schilter, et al., ChemCrow: Augmenting large-language models with chemistry tools, in: *Advances in Neural Information Processing Systems*, Vol. 36, 2023.
- [13] Y. Ma, Z. Liu, L. P. Liang, et al., SciAgent: Tool-augmented language models for scientific reasoning, *arXiv preprint arXiv:2402.11451* (2024).

- [14] A. Logg, K.-A. Mardal, G. N. Wells, et al., Automated Solution of Differential Equations by the Finite Element Method: The FEniCS Book, Springer, 2012. doi:10.1007/978-3-642-23099-8.
- [15] I. A. Barrata, J. P. Dean, J. S. Dokken, et al., DOLFINx: The next generation FEniCS problem solving environment, Zenodo (2023). doi:10.5281/zenodo.10447666.
- [16] P. Bauer, P. Henning, J. Schaefer, Large language models as automatic generators of FEniCS code for solving partial differential equations, arXiv preprint arXiv:2312.09801 (2023).
- [17] W. Jiang, K. Chen, M. Wang, et al., LLM4FEM: Leveraging large language models for finite element method, arXiv preprint arXiv:2405.03719 (2024).
- [18] R. Deotale, A. Srinivasan, M. Golestanian, Y. Tian, T. Zhang, P. Vlachos, H. Gomez, ALL-FEM: Agentic large language models fine-tuned for finite element methods, Computer Methods in Applied Mechanics and Engineering 457 (2026) 118985. doi:10.1016/j.cma.2026.118985.
- [19] N. Mudur, H. Cui, S. Venugopalan, P. Raccuglia, M. P. Brenner, P. Norgaard, FEABench: Evaluating language models on multiphysics reasoning ability, arXiv preprint arXiv:2504.06260 (2025).
- [20] LangChain AI, LangGraph: Build stateful, multi-actor applications with LLMs (2024).
URL <https://github.com/langchain-ai/langgraph>

- [21] Q. Wu, G. Bansal, J. Zhang, et al., AutoGen: Enabling next-generation LLM applications via multi-agent conversation, in: Proceedings of EMNLP Industry Track, 2023.
- [22] J. Moura, CrewAI: Framework for orchestrating role-playing, autonomous AI agents (2024).
URL <https://github.com/joaomdmoura/crewai>
- [23] X. Liao, H. Zhang, Y. Chen, Retrieval-augmented generation for engineering design documentation, arXiv preprint arXiv:2307.04512 (2023).
- [24] Y. Gao, S. Liu, P. Shi, J. Lin, Retrieval-augmented code generation for universal information extraction, arXiv preprint arXiv:2311.02555 (2023).
- [25] Z. Yang, W. Li, P. Zhang, Simulation parameter suggestion via retrieval-augmented generation, arXiv preprint arXiv:2403.09512 (2024).
- [26] S.-Q. Yan, J.-C. Gu, Y. Zhu, Z.-H. Ling, Corrective retrieval augmented generation, in: Proceedings of the Conference on Empirical Methods in Natural Language Processing (EMNLP), 2024, arXiv:2401.15884.
- [27] P. Anokhin, N. Kornaev, A. Babkin, A. I. Panov, AriGraph: Learning knowledge graph world models with episodic memory for LLM agents, in: Advances in Neural Information Processing Systems (NeurIPS), 2024, arXiv:2407.04363.
- [28] V. Venugopal, S. Sahoo, G. Agastya, et al., MatKG: The largest knowledge graph in applied materials science, arXiv preprint arXiv:2209.11632 (2022).

- [29] C. W. Andersen, R. Armiento, E. Blokhin, et al., OPTIMADE: Towards an open database for computational materials science, *Scientific Data* 8 (1) (2021) 217. doi:10.1038/s41597-021-00974-z.
- [30] M. J. Buehler, Generative retrieval-augmented ontologic graph and multiagent strategies for interpretive large language model-based materials design, *ACS Engineering Au* 4 (2) (2024) 241–277. doi:10.1021/acsengineeringau.3c00058.
- [31] C. Geuzaine, J.-F. Remacle, Gmsh: A 3-d finite element mesh generator with built-in pre- and post-processing facilities, *International Journal for Numerical Methods in Engineering* 79 (11) (2009) 1309–1331. doi:10.1002/nme.2579.
- [32] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, A. Taylor, Cypher: An evolving query language for property graphs, in: *Proceedings of the 2018 International Conference on Management of Data (SIGMOD)*, 2018, pp. 1433–1445. doi:10.1145/3183713.3190657.
- [33] Z. Nussbaum, J. X. Morris, B. Duderstadt, A. Mulyar, Nomic embed: Training a reproducible long context text embedder, *arXiv preprint arXiv:2402.01613* (2024).
- [34] IBM Research, Docling: Document processing for AI (2024). URL <https://github.com/DS4SD/docling>
- [35] ASME, Guide for verification and validation in computational solid me-

902 chanics, Tech. Rep. ASME V&V 10-2006, American Society of Mechanical
903 Engineers (2006).

904 [36] E. B. Wilson, Probable inference, the law of succession, and statistical
905 inference, *Journal of the American Statistical Association* 22 (158) (1927)
906 209–212. doi:10.1080/01621459.1927.10502953.

907 [37] J. Cohen, *Statistical Power Analysis for the Behavioral Sciences*, 2nd
908 Edition, Lawrence Erlbaum Associates, 1988.

909 [38] H. Chen, L. Mangani, G. Casas, OpenFOAMGPT 2.0: End-to-end,
910 trustworthy automation for computational fluid dynamics, arXiv preprint
911 arXiv:2504.19338 (2025).

912 [39] Y. Chen, X. Zuo, Y. Yang, et al., MetaOpenFOAM: An LLM-based
913 multi-agent framework for CFD, arXiv preprint arXiv:2407.21320 (2024).

914 [40] S. Hong, M. Zhuge, J. Chen, X. Zheng, Y. Cheng, C. Zhang, J. Wang,
915 Z. Wang, S. K. S. Yau, Z. Lin, L. Zhou, C. Ran, L. Xiao, C. Wu, J. Schmid-
916 huber, MetaGPT: Meta programming for a multi-agent collaborative
917 framework, arXiv preprint arXiv:2308.00352 (2024).

918 [41] R. Cao, M. Chen, J. Chen, Z. Cui, Y. Feng, B. Hui, Y. Jing, K. Li, M. Li,
919 J. Lin, Z. Ma, K. Shum, X. Wang, J. Wei, J. Yang, J. Zhang, L. Zhang,
920 Z. Zhang, W. Zhao, F. Zhou, Qwen3-coder-next technical report, arXiv
921 preprint arXiv:2603.00729 (2026).

922 [42] Meta AI, The Llama 4 herd: The beginning of a new era
923 of natively multimodal AI innovation, [https://ai.meta.com/blog/
924 llama-4-multimodal-intelligence/](https://ai.meta.com/blog/llama-4-multimodal-intelligence/), accessed 2026-04-15 (2025).

925 Appendix A. Full h-Refinement Data

926 Table A.11 reports the full mesh-refinement study behind the convergence
927 rates summarised in table 2: L^2 and L^∞ error norms at every resolution
928 $N \in \{8, 16, 32, 64, 128\}$ for all three benchmark cases.

929 The two norms are computed differently, and for one case they disagree
930 by several orders of magnitude. $\|e\|_{L^2}$ is assembled by quadrature over each
931 element, so it measures error throughout the domain, including between nodes.
932 $\|e\|_{L^\infty}$ is evaluated only at the degrees of freedom. For the constant-source
933 (1D-like Poisson) case the P1 solution is *nodally exact*, which is a standard
934 superconvergence result, so the nodal norm sits at machine precision ($\approx 10^{-10}$)
935 at every resolution while the integrated norm decays at $\mathcal{O}(h^2)$ as expected.
936 Convergence rates are therefore fitted from $\|e\|_{L^2}$ alone; a rate fitted from
937 nodal values would say nothing useful about this case.

Table A.11: Detailed convergence data: error norms at each mesh resolution. $\|e\|_{L^2}$ is integrated over each element with quadrature of degree 8, so it captures the error *between* nodes; $\|e\|_{L^\infty}$ is evaluated at the degrees of freedom only. For the constant-source case the P1 solution is nodally exact, so $\|e\|_{L^\infty}$ sits at machine precision while $\|e\|_{L^2}$ shows the expected $\mathcal{O}(h^2)$ interpolation error. The two norms measure different things, and only the integrated norm is used to fit convergence rates.

Case	N	DOFs	$\ e\ _{L^2}$	$\ e\ _{L^\infty}$
2D Steady-State Linear Profile	8	81	4.22e-11	1.12e-10
	16	289	3.65e-09	1.44e-08
	32	1,089	3.89e-10	1.11e-09
	64	4,225	1.10e-09	3.00e-09
	128	16,641	2.47e-09	6.40e-09
2D Transient Fourier Mode Decay	8	81	4.38e-03	4.88e-03
	16	289	9.48e-04	2.64e-03
	32	1,089	2.39e-04	6.62e-04
	64	4,225	5.98e-05	1.65e-04
	128	16,641	1.50e-05	4.13e-05
2D Steady-State with Constant Source (1D-like)	8	81	1.43e-03	1.12e-10
	16	289	3.57e-04	2.28e-11
	32	1,089	8.91e-05	1.22e-10
	64	4,225	2.23e-05	1.83e-10
	128	16,641	5.57e-06	2.68e-10